Scalable LLM Math Reasoning Acceleration with Low-rank Distillation

Harry Dong* CMU
Bilge Acun† Meta
Beidi Chen* CMU
Yuejie Chi*† CMU, Meta

May 28, 2025

Abstract

Due to long generations, large language model (LLM) math reasoning demands significant computational resources and time. While many existing efficient inference methods have been developed with excellent performance preservation on language tasks, they often severely degrade math performance. In this paper, we propose `Caprese`, a resource-efficient distillation method to recover lost capabilities from deploying efficient inference methods, focused primarily in feedforward blocks. With original weights unperturbed, roughly 1% of additional parameters, and only 20K synthetic training samples, we are able to recover much if not all of the math capabilities lost from efficient inference for thinking LLMs and without harm to language tasks for instruct LLMs. Moreover, `Caprese` slashes the number of active parameters (∼2B cut for Gemma 2 9B and Llama 3.1 8B) and integrates cleanly into existing model layers to reduce latency (>11% reduction to generate 2048 tokens with Qwen 2.5 14B) while encouraging response brevity.

1 Introduction

With the increasing capabilities of large language models (LLMs) [VSP+17], evaluations are also becoming increasingly sophisticated, typically involving multi-step reasoning such as in math problem solving. These tasks tend to demand long generation which drives up latency, making efficiency a dire issue. Fortunately, many efficient LLM inference algorithms have shown great promise, slashing expensive computational bottlenecks with little damage to the original performance on a variety of language-based tasks like reading comprehension and summarization. *However, for math reasoning tasks, many efficient inference algorithms begin to break down, decimating performance, despite their robustness in language settings.* Thus, there is a need to design efficient inference algorithms that simultaneously maintain language and math capabilities.

One of the main differences between math reasoning and many language tasks is the generation length. Reasoning typically involves long generation due to improved performance from chain-of-thought (CoT) [WWS+22], which often surpasses the length of the input query. However, CoT performance falls apart with efficient algorithms that introduce approximation errors which build up over time. For instance, deploying CATS [LLZ+24], a sparse thresholding method, on Gemma 2 2B [TRP+24] has little impact on language generation performance, but knocks GSM8K [CKB+21] accuracy *from 51.02% to 34.42%* (Table 1). Similarly, for thinking models: applying GRIFFIN [DCC24], an adaptive structured pruning method, on the first half of DeepSeek-R1-Distill-Qwen 1.5B [GYZ+25] drops MATH-500 [LKB+23] accuracy *from 79.40% to 42.00%* (Table 2). Moreover, since generation is much more demanding computationally than prefill (e.g., >150× slower to generate than prefill 1K tokens on one GPU with Llama 3.1 8B), there is a dire need to make long reasoning CoTs efficient, which is made more apparent with the rise of test-time scaling. Thus, this poses two key inference challenges with math reasoning:

*Department of Electrical and Computer Engineering, Carnegie Mellon University, USA; Emails: {harryd,beidic,yuejiec}@andrew.cmu.edu.
†FAIR at Meta

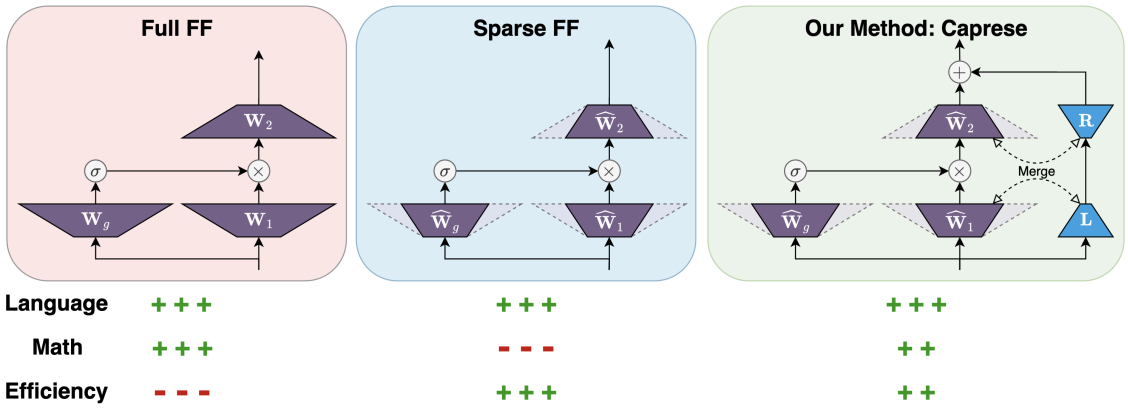

Figure 1: A full FF block will contain the complete performance of the original model without any benefit to efficiency. Sparse FF algorithms can be very efficient by using subsets of the FF block but harm math performance. Our method, `Caprese`, uses a sparse FF algorithm and a small distilled low-rank linear layer, which can be merged with existing FF weights, for highly performative inference in language and math settings while being efficient. Layers are drawn as trapezoids to highlight the relative size of inputs and outputs.

1. *Instability*: Long CoTs can be sensitive to mistakes. Even single token mistakes can sometimes drive the generation trajectory off course, leading to an incorrect response [ZCX+24]. Hence, efficient inference algorithms should respect the delicacy of CoT.

2. *Inefficiency*: Due to the reliance on CoT, which tend to lead to long generation, the already computationally expensive LLM autoregressive decoding process is accentuated. This problem is further exaggerated as CoTs scale in length in thinking models [GYZ+25] and as the number of repeated generations scale to elicit higher quality answers [BJE+24, WSL+24, SLXK24].

An ideal method should be efficient, performance preserving, and easy to integrate. Thankfully, low-rank structure in the feedforward (FF) output features can help make this possible. (We choose to focus on FF blocks since, contributing around 2/3 of an LLM's parameters and over 50% of the generation latency.) Because of the success of works that exploit contextual sparsity in FF blocks on language tasks for efficiency, like CATS and GRIFFIN, we peek into the residuals of FF sparsity-based efficient methods. Using an oracle top-k filter on FF nonlinearity output magnitudes, we observe huge reductions in error with a low-rank approximation to the FF output residuals (Figure 2). Since FF intermediate feature sizes can be on the order of 10^5, adding 256 for a low-rank approximation is comparatively mundane. *This observation motivates us to estimate the residual from sparse FF methods with low-rank layers.*

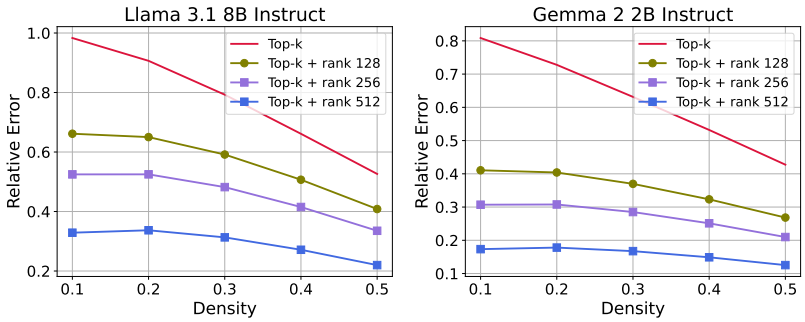

Figure 2: Average relative FF output error of generated tokens with varying top-k densities and low-rank approximations. The density is the fraction of non-zero intermediate FF features maintained by top-k. Samples consist of 16 random MATH prompts and generations by the original model. *A relatively small low-rank approximation to the top-k residual dramatically reduces error much more effectively than just increasing density.*

We introduce `Caprese`[1] (**CAP**ability **RE**covery with **S**calable **E**fficiency) to learn FF residuals from *any* sparse FF algorithm using small low-rank linear layers for improved performance (Figure 1). Through distillation of math knowledge into these low-rank layers, `Caprese` is able to overcome both challenges of math reasoning and makes significant progress towards an ideal efficient method:

1. **Performance Enhancement:** Demonstrated across a variety of models (e.g., instruct and thinking models), `Caprese` *recovers much if not all of the math performance lost from deploying a sparse FF algorithm without harming language tasks.* Also, `Caprese` maintains its performance benefit when applying different techniques of test-time scaling. For example, scaling generations with `Caprese` on Llama 3.2 3B Instruct improves Pass@100 by 7.0% from the original model while using 15.8% less compute.

2. **Efficiency:** Due to the ability to combine our low-rank layers with existing FF layers, `Caprese` reduces generation latency. For instance, `Caprese` reduces latency by 11.7% when generating 2048 tokens using DeepSeek-R1-Distill-Qwen 14B with GRIFFIN.

3. **Low Budget:** The distillation process of `Caprese` is cheap since we are able to see significant gains in math performance by training on only 20K synthetic math samples. Moreover, with a low-rank layer size of only 256, this equates to adding roughly 0.8% additional parameters into Llama 3.1 8B and Gemma 2 9B, dwarfed by savings in active parameters (∼2B cut for both models with `Caprese` (CATS)).

While `Caprese` is not restricted to math tasks, we focus on math in this paper, as current efficient sparse methods can obliterate an LLM's math capabilities. Future work can investigate its benefit to other domains.

The remainder of this paper is organized as follows. Section 2 covers background on related works and describes GRIFFIN and CATS, two efficient sparse FF algorithms that serve as baselines. Section 3 details `Caprese`'s architecture and distillation procedure. Then, we showcase the strong performance of `Caprese` in Section 4: instruct LLM inference (Section 4.1), scaling generation outputs (Section 4.2), and scaling generation length with thinking LLMs (Section 4.3). Finally, we quantify latency improvements in Section 4.4.

2 Background

In this section, we provide an overview of the efficient LLM inference literature that `Caprese` builds upon with in-depth descriptions of GRIFFIN and CATS which we use as baselines and some recent developments for test-time scaling.

2.1 Related Works

Efficient Inference for FF Blocks. To improve the efficiency of FF blocks in LLMs, various methods leverage existing sparse structures in FF features [GSBL20, DLBZ22, LYB+22, DCC23, LWD+23]. Pruning sets a parameter subset to zero to enforce static sparsity [LDS89,SLBK23,FA23,MFW23]. Mixtures of experts (MoEs) adaptively select predefined parameter subsets in FF blocks, though they tend to require significant fine-tuning or training from scratch [JJNH91, ZLL+21, DDZ+24, LWD+23]. Similar to MoEs, another line of work aims at exploiting contextual sparsity to dynamically select FF neurons for each input without training. GRIFFIN [DCC24] is a calibration-free method that uses FF activations from the prefill phase to adaptively prune FF neurons for generation. CATS [LLZ+24] uses hard thresholding to skip computation in parts of the FF block with a custom kernel. More details of GRIFFIN and CATS can be found in Section 2.2.

Shorter CoTs. Several works aim to improve reasoning efficiency by directly reducing the number of generated tokens [RG24, NRS+24, AZ25, ABH25, XXZH25, QYS+25]. Since we aim to reduce the per-token latency, this line of research to encourage brevity is orthogonal to ours and could be used alongside our method.

[1] Our method's components are modular with easy preparation like a Caprese salad.

Test-time Scaling. To enhance LLM performance, the priority has historically been to scale training with more data and bigger models [KMH+20, HBM+22]. More recently, there is an increasing effort towards test-time scaling to boost performance on difficult tasks like math. Two ways of test-time scaling that have seen great success. First, for a single prompt, multiple responses can be sampled from the LLM [BJE+24, SLXK24, WSL+24, MSE24, SHZ+24]. This increases the probability that a desired response lies in the pool of responses. While this method of scaling is highly parallelizable, it also relies on a verifier model to select the best one. Second, CoTs can be lengthened for each response [GYZ+25]. By extrapolating the success of CoTs to extreme lengths (e.g., DeepSeek-R1 generates up to 32K tokens per response), the accuracy of the final answer improves significantly, but this method has low parallelizability due to the autoregressive nature of generation.

2.2 Training-free & Adaptively Sparse FF Methods

We now briefly describe the inner workings of GRIFFIN and CATS. Let $\boldsymbol{X} \in \mathbb{R}^{S\times D}$ be the input into the FF block during the prefill phase with sequence length S and feature size D. Define the FF block as $\mathrm{FF}(\boldsymbol{X}) = \mathrm{FF}_2(\mathrm{FF}_1(\boldsymbol{X}))$ such that

$$\boldsymbol{Z} = \mathrm{FF}_1(\boldsymbol{X}) = \sigma(\boldsymbol{X}\boldsymbol{W}_g) \odot \boldsymbol{X}\boldsymbol{W}_1, \tag{1}$$

$$\mathrm{FF}_2(\boldsymbol{Z}) = \boldsymbol{Z}\boldsymbol{W}_2, \tag{2}$$

where $\boldsymbol{W}_g, \boldsymbol{W}_1 \in \mathbb{R}^{D\times D_{\mathrm{FF}}}$, $\boldsymbol{W}_2 \in \mathbb{R}^{D_{\mathrm{FF}}\times D}$, σ is a nonlinear function, $\odot$ is an element-wise multiplication operator, and D_{FF} is the FF intermediate feature size. Typically, $D_{\mathrm{FF}} \gg D$. Although recent LLMs use this architecture, for LLMs without gated functions (e.g., OPT [ZRG+22]), $\boldsymbol{X}\boldsymbol{W}_1$ can be removed. Bias terms are omitted for brevity.

GRIFFIN. GRIFFIN [DCC24] adaptively prunes columns and rows in the FF block weights, using FF activation statistics from the prefill phase, namely the flocking patterns. GRIFFIN calculates $[\overline{\boldsymbol{Z}}]_i = |[\boldsymbol{Z}]_i|/\|[\boldsymbol{Z}]_i\|_2$ for each token index i, followed by an aggregation across the FF feature axis: $[\boldsymbol{s}]_j = \|[\overline{\boldsymbol{Z}}]_{\cdot,j}\|_2$. The result $\boldsymbol{s} \in \mathbb{R}^{D_{\mathrm{FF}}}$ gives a metric to perform top-k selection across corresponding columns of $\boldsymbol{W}_g$ and $\boldsymbol{W}_1$, and rows of $\boldsymbol{W}_2$ to produce $\widehat{\boldsymbol{W}}_g, \widehat{\boldsymbol{W}}_1 \in \mathbb{R}^{D\times k}$, and $\widehat{\boldsymbol{W}}_2 \in \mathbb{R}^{k\times D}$. Then, for the generation phase, the following FF block is used for input $\boldsymbol{x} \in \mathbb{R}^D$:

$$\widehat{\boldsymbol{z}} = \widehat{\mathrm{FF}}_1(\boldsymbol{x}) = \sigma(\boldsymbol{x}\widehat{\boldsymbol{W}}_g) \odot \boldsymbol{x}\widehat{\boldsymbol{W}}_1, \tag{3}$$

$$\widehat{\mathrm{FF}}_2(\widehat{\boldsymbol{z}}) = \widehat{\boldsymbol{z}}\widehat{\boldsymbol{W}}_2. \tag{4}$$

The compressed FF blocks are fixed throughout generation but are dynamic across prompts.

CATS. CATS [LLZ+24] uses hard thresholding to skip computation in part of the FF block. Letting T_τ be the hard thresholding function with threshold τ, CATS computes

$$\widehat{\boldsymbol{z}} = \widehat{\mathrm{FF}}_1(\boldsymbol{x}) = T_\tau(\sigma(\boldsymbol{x}\boldsymbol{W}_g)) \odot \boldsymbol{x}\boldsymbol{W}_1, \tag{5}$$

$$\widehat{\mathrm{FF}}_2(\widehat{\boldsymbol{z}}) = \widehat{\boldsymbol{z}}\boldsymbol{W}_2. \tag{6}$$

The weights $\boldsymbol{W}_1$ and $\boldsymbol{W}_2$ should be sparsified into $\widehat{\boldsymbol{W}}_1$ and $\widehat{\boldsymbol{W}}_2$, respectively, based on the non-zero entries of $T_\tau(\sigma(\boldsymbol{x}\boldsymbol{W}_g))$ for latency improvement, which can vary from token to token and require a custom kernel for wall clock speed-up. The parameter τ is calibrated to be a desired percentile on a dataset. In this paper, to avoid calibration, we set τ based on prefill features and only threshold during the generation phase, analogous to GRIFFIN.

3 Method: `Caprese`

Motivated by the low-rank structure of residuals in Figure 2, we introduce `Caprese` which distills approximation errors in embeddings into low-rank linear layers in FF blocks. See Figure 1 for an illustration of our method.

3.1 Layer-wise Distillation

Inference efficiency algorithms often introduce feature approximation errors in favor of faster generation, which we mitigate with distillation. Let the efficient and approximate FF block be $\widehat{\text{FF}}(\boldsymbol{x}) = \widehat{\text{FF}}_2(\widehat{\text{FF}}_1(\boldsymbol{x}))$. In our design of `Caprese`, we do not constrain $\widehat{\text{FF}}$ to be any specific method or architecture. For instance, $\widehat{\text{FF}}$ could be an FF block with GRIFFIN [DCC24] or CATS [LLZ+24]. Then, the error is

$$\|\text{FF}(\boldsymbol{x}) - \widehat{\text{FF}}(\boldsymbol{x})\|_2^2.$$

We choose to reduce this residual with a low-rank linear layer, meaning we want to solve

$$\min_{\boldsymbol{L},\boldsymbol{R}} \frac{1}{|\mathcal{X}|} \sum_{\boldsymbol{x}\in\mathcal{X}} \|\text{FF}(\boldsymbol{x}) - \widehat{\text{FF}}(\boldsymbol{x}) - \boldsymbol{x}\boldsymbol{L}\boldsymbol{R}\|_2^2 \tag{7}$$

for input set $\mathcal{X}$, $\boldsymbol{L} \in \mathbb{R}^{D\times r}$, and $\boldsymbol{R} \in \mathbb{R}^{r\times D}$ where $r \ll D_{\text{FF}}$. The optimal solution can be computed analytically since this is a reduced rank regression problem, but the size of $|\mathcal{X}|$ and D may make it prohibitively expensive. Therefore, we opt to learn $\boldsymbol{L}$ and $\boldsymbol{R}$ independently for every FF block (i.e., previous FF blocks are assumed to be from the original model), allowing for parallel layer-wise training. This takes inspiration from LESS [DYZ+24] which uses layer-wise training on attention residuals for key-value cache compression. Each $\boldsymbol{R}$ is initialized as a zero matrix since the efficient approximation is assumed to be of good quality, and original model weights are frozen.

Even if the layer-wise MSE is minimized, this does not guarantee the final outputs closely resemble the original model due to error accumulation. Hence, we also distill end-to-end (E2E) to further improve the performance.

3.2 End-to-end Distillation

Using the learned low-rank layers as an initialization, we put them all together to distill the final model embedding before the linear head into the efficient model, again using MSE:

$$\min_{(\boldsymbol{L}_i,\boldsymbol{R}_i)_{i=1,\dots,L}} \frac{1}{|\mathcal{X}|} \sum_{\boldsymbol{x}\in\mathcal{X}} \|\text{M}(\boldsymbol{x}) - \text{M}_{\text{student}}(\boldsymbol{x})\|_2^2 \tag{8}$$

where M and $\text{M}_{\text{student}}$ are the original LLM and efficient LLM with distillation layers, respectively, excluding the final linear head. L is the number of layers in the the model. All original weights are frozen, so the only tunable parameters are the $\boldsymbol{L}$ and $\boldsymbol{R}$ of each FF block.

3.3 Parallel Inference Computation

The computation of $\boldsymbol{x}\boldsymbol{L}\boldsymbol{R}$ can be done in parallel with the original FF operations. In fact, $\boldsymbol{L}$ and $\boldsymbol{R}$ can be concatenated with the up and down projection matrices, respectively. In other words, the `Caprese` FF block is $\widehat{\text{FF}}^+(\boldsymbol{x}) = \widehat{\text{FF}}_2^+(\widehat{\text{FF}}_1^+(\boldsymbol{x}))$, such that

$$\hat{\boldsymbol{z}}^+ = \widehat{\text{FF}}_1^+(\boldsymbol{x}) = \left[\sigma(\boldsymbol{x}\widehat{\boldsymbol{W}}_g) \quad \boldsymbol{1}_r\right] \odot \boldsymbol{x}\left[\widehat{\boldsymbol{W}}_1 \quad \boldsymbol{L}\right], \tag{9}$$

$$\widehat{\text{FF}}_2^+(\hat{\boldsymbol{z}}^+) = \hat{\boldsymbol{z}}^+\left[\widehat{\boldsymbol{W}}_2^\top \quad \boldsymbol{R}^\top\right]^\top, \tag{10}$$

where $\boldsymbol{1}_r$ is a one-vector with length r. In practice, to save memory and time, we do not materialize $\boldsymbol{1}_r$ but directly assign the product with $\sigma(\boldsymbol{x}\widehat{\boldsymbol{W}}_g)$ to corresponding entries of $\boldsymbol{x}\widehat{\boldsymbol{W}}_1^+$. Recall that the prefill stage still just uses the original model.

3.4 Training Details

We set the inner dimension of our low-rank layer to $r = 256$ (to see the effect of different r, see Appendix A). In comparison to the enormous inner dimension of FF layers (e.g., $D_{\text{FF}} = 14336$ for Llama 3.1 8B and

Model	GSM8K	MATH	CoQA	QASPER
Llama 3.1 8B Instruct	27.90	13.94	63.88	15.16
GRIFFIN	17.44	6.16	63.37	12.63
Layer-wise Caprese	27.14	9.72	65.05	14.21
E2E Caprese	**40.49**	**12.64**	**65.50**	**15.35**
CATS	40.56	11.80	58.85	12.26
Layer-wise Caprese	37.68	13.66	63.92	14.34
E2E Caprese	**51.86**	**13.88**	**64.27**	**14.42**
Llama 3.2 1B Instruct	22.44	10.66	55.43	14.43
GRIFFIN	7.13	5.42	56.05	**14.11**
Layer-wise Caprese	13.72	6.62	56.07	13.40
E2E Caprese	**21.00**	**8.44**	**56.55**	13.88
CATS	19.18	7.54	54.40	13.88
Layer-wise Caprese	18.65	8.28	55.58	**14.35**
E2E Caprese	**20.39**	**9.04**	**56.12**	13.75
Llama 3.2 3B Instruct	51.55	14.32	63.95	12.45
GRIFFIN	28.96	10.98	64.52	**12.52**
Layer-wise Caprese	40.18	13.70	64.33	11.60
E2E Caprese	**44.66**	**16.96**	**64.83**	12.35
CATS	41.24	12.04	58.87	11.36
Layer-wise Caprese	45.49	13.62	60.53	12.26
E2E Caprese	**46.85**	**14.40**	**61.72**	**12.36**
Gemma 2 2B Instruct	51.02	16.06	63.77	10.96
GRIFFIN	33.74	11.32	63.28	**11.07**
Layer-wise Caprese	42.53	12.32	**63.77**	10.75
E2E Caprese	**48.14**	**13.70**	63.37	11.05
CATS	34.42	10.56	61.53	10.11
Layer-wise Caprese	**46.32**	13.90	61.92	10.82
E2E Caprese	46.17	**14.16**	**63.92**	**11.03**
Gemma 2 9B Instruct	78.17	27.64	63.78	9.91
GRIFFIN	59.21	25.22	63.82	**10.14**
Layer-wise Caprese	**76.72**	**25.84**	64.20	9.82
E2E Caprese	76.65	25.30	**64.42**	9.92
CATS	76.50	27.32	64.37	9.78
Layer-wise Caprese	**77.18**	**28.16**	64.52	**10.46**
E2E Caprese	**77.18**	28.00	**64.87**	10.02

Table 1: Instruct models' 0-shot accuracies on mathematical reasoning (GSM8K and MATH) and language generation tasks (CoQA and QASPER). FF sparsity is set to 50%, and $r = 256$.

Gemma 2 9B), our choice of r is relatively miniscule, adding only $\sim 1\%$ new parameters for all tested models (Appendix B).

We use a 20K subset of a common synthetic math training set for training. For a fair comparison, the same subset is used for both layer-wise and E2E distillation for every model. Each training sample is prepended with a CoT instruction: "Please reason step by step." At test time, the actual instructions may be vastly different. Layer-wise and E2E training consists of 20 epochs and 3 epochs, respectively. Training and inference are done in BF16. Hyperparameters are listed in Appendix E.

4 Experiments

We showcase the effectiveness of `Caprese` at recovering much, if not all, of the math performance lost from efficient inference algorithms without sacrificing efficiency or performance on language tasks. In the following section (Section 4.1), we observe better math performance for instruct LLMs without losing their generalizability. Then, we study `Caprese`'s benefit on two axes of test-time scaling: number of generations

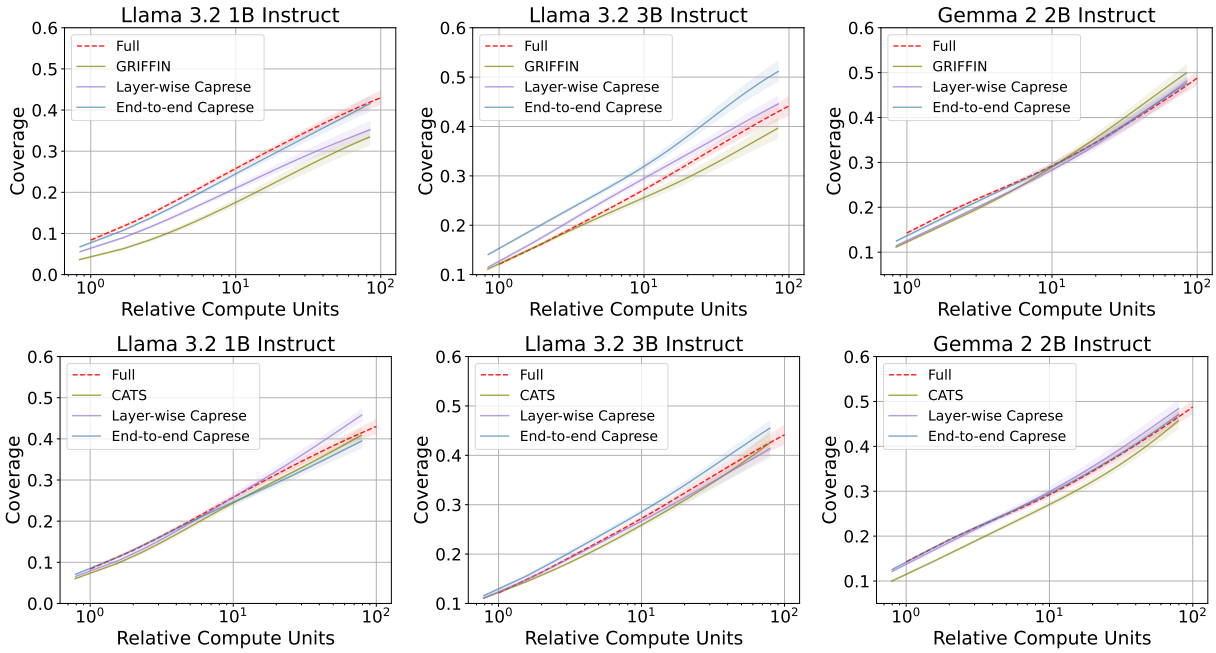

Figure 3: Coverage and standard deviation of 140 samples from MATH as the number of generation attempts, N, scales. We define Relative Compute Units $= N \times A$ where A is the fraction of total parameters activated per input. FF sparsity set to 50%. Best viewed zoomed.

(Section 4.2) and generation length with thinking models (Section 4.3). Finally, we highlight `Caprese`'s latency improvements in Section 4.4. When ambiguous, we denote `Caprese` (CATS) to be our method with CATS as the underlying sparse method and similarly for GRIFFIN. Otherwise, we use "`Caprese`" for brevity. Unless specified, the FF intermediate feature sparsity is set at 50%. *NOTE: For a more meaningful baseline, we only apply GRIFFIN to the first half of the model for all experiments since we observed much steeper drops in math performance if used for all layers. Regardless, `Caprese` still improves the performance of GRIFFIN on all layers. CATS is applied to all layers.*

4.1 Instruct Models

We first looking into different instruct LLMs which are performative on a variety of non-reasoning tasks and decent reasoning performance. We show `Caprese` is able to preserve math performance without sacrificing performance on pure language tasks like question answering. In more detail, we test on the Llama 3 family [DJP+24] and Gemma 2 family [TRP+24] of models on zero-shot GSM8K [CKB+21], MATH [HBK+21], CoQA [RCM19], and QASPER [DLB+21], using LM Evaluation Harness [GTA+23] and CoT prompts for math tasks.

Table 1 shows that *`Caprese` is able to preserve most if not all of the math capabilities in the original models without damaging performance on the language tasks, despite the distillation dataset being all math.* In most cases, CATS and GRIFFIN severely harm GSM8K and MATH accuracy, but `Caprese` is able to effectively recover the lost performance. With the exception of Gemma 2 9B Instruct where the layer-wise distillation appears to be better, E2E `Caprese` is the most performative in the majority of math scenarios, suggesting a benefit beyond minimizing local error. `Caprese`'s performance is consistent at different sparsity levels and r (Appendix A). The best performers in CoQA and QASPER are a toss-up, but all methods have little impact on the accuracy of these tasks (the main purpose of these tasks is to show no degradation in language-related cases). CATS and `Caprese` considerably improve GSM8K performance of Llama 3.1 8B Instruct, which has trouble following the prompt template with zero-shot.

Model	MATH-500	AIME 2024	AMC 2023
DeepSeek-R1-Distill-Qwen 1.5B	79.40	31.11 ± 1.92	57.50 ± 5.00
GRIFFIN	42.00	2.22 ± 1.92	16.67 ± 1.44
Layer-wise Caprese	47.20	3.33 ± 0.00	27.50 ± 2.50
E2E Caprese	**60.40**	**6.67** ± 3.33	**35.83** ± 7.22
CATS	72.00	14.44 ± 5.09	38.33 ± 3.82
Layer-wise Caprese	73.80	16.67 ± 3.33	46.67 ± 2.89
E2E Caprese	**74.80**	**21.11** ± 1.92	**48.33** ± 3.82
DeepSeek-R1-Distill-Qwen 7B	90.20	51.11 ± 3.85	76.67 ± 2.89
GRIFFIN	80.60	20.00 ± 3.33	58.33 ± 3.82
Layer-wise Caprese	84.80	27.78 ± 1.92	60.83 ± 1.44
E2E Caprese	**85.40**	**28.89** ± 6.94	**70.00** ± 5.00
CATS	89.20	**37.78** ± 5.77	62.50 ± 2.50
Layer-wise Caprese	87.00	35.56 ± 5.09	65.83 ± 7.22
E2E Caprese	**90.00**	32.22 ± 5.09	**79.17** ± 5.77
DeepSeek-R1-Distill-Qwen 14B	92.80	63.33 ± 3.33	90.83 ± 2.89
GRIFFIN	89.80	30.00 ± 6.67	79.17 ± 2.89
Layer-wise Caprese	**90.80**	**38.89** ± 1.92	**87.50** ± 5.00
E2E Caprese	89.20	37.78 ± 10.72	82.50 ± 6.61
CATS	**92.80**	57.78 ± 1.92	**91.67** ± 5.20
Layer-wise Caprese	91.00	**60.00** ± 5.77	88.33 ± 1.44
E2E Caprese	92.00	58.89 ± 11.71	85.83 ± 3.82

Table 2: Thinking models' 0-shot accuracies on math reasoning tasks. Sparsity is set at 50%, and $r = 256$. AIME 2024 and AMC 2023 columns also include the sample standard deviation across 3 runs. Further improvements with reselection are shown in Table 3.

4.2 Scaling Best-of-N: Coverage

Now, we see the generalizability of `Caprese` on the first axis of test-time scaling: sampling multiple responses and selecting the best one, known as Best-of-N (BoN). Increasing the number of generations per prompt, N, is one way to scale inference compute for improved performance, as the probability of generating a correct answer increases. To evaluate, we find the coverage (Pass@K) on 140 samples from MATH. We use an oracle verifier to accurately assess the quality of the pool of generated responses. To combat the high variance when K is close to N, we calculate the average coverage for $K = 1, \ldots, 100$ across 10 independent pools of 100 generations for each sample. *Factoring in the saved compute, E2E `Caprese` is able to have similar or better coverage scaling compared to the original model*, as shown in Figure 3. Notably, E2E `Caprese` (GRIFFIN) on Llama 3.2 3B Instruct improves Pass@100 by 7.0% from the original model while using 15.8% less compute.

4.3 Scaling Length: Thinking Models

Thinking models provide another axis of test-time scaling by augmenting the CoT length before giving a final answer. Because this entails very long generation lengths, it is critical that error accumulation across tokens be minimized. We test on DeepSeek-R1-Distill-Qwen models [YYZ+24, GYZ+25] using the prompt templates and configurations from Open R1 [Fac25] (temperature and top-p set to 0.6 and 0.95, respectively). For these models, we evaluate zero-shot performance on MATH-500 [HBK+21, LKB+23], AIME 2024, and AMC 2023 for a max generation length of 32768. Due to the small number of problems in AIME 2024 (30 problems) and AMC 2023 (40 problems), we evaluate these 3 independent times, reporting the average accuracy and sample standard deviation.

From Table 2, `Caprese` is the most performative in most cases. All methods generally appear to be more robust as the model size increases. For instance, CATS and GRIFFIN had very little impact on DeepSeek-R1-Distill-Qwen 14B's accuracy on MATH-500, with `Caprese` performing similarly. By far, AIME 2024 is the most challenging and noisy task (due to its difficult and small dataset), with severe degradation from CATS and GRIFFIN and partial recovery using our method. In contrast, AMC 2023 results are less noisy

Model	No Reselect	$\rho = 1024$	$\rho = 256$	Full
DeepSeek-R1-Distill-Qwen 1.5B	35.83 ± 7.22	**40.83** ± 5.77	40.00 ± 7.50	57.50 ± 5.00
DeepSeek-R1-Distill-Qwen 7B	70.00 ± 5.00	68.17 ± 1.44	**74.17** ± 5.20	76.67 ± 3.82
DeepSeek-R1-Distill-Qwen 14B	82.50 ± 6.61	87.50 ± 4.33	**89.17** ± 3.82	90.83 ± 2.89

Table 3: E2E `Caprese` AMC 2023 accuracies and standard deviations when recalculating the GRIFFIN pruning metric during generation. ρ is the rate at which we reselect pruned neurons in terms of generated tokens. No reselection and the full model are special cases where $\rho = \infty$ and $\rho = 1$, respectively.

and regained performance from our method is much more substantial. *In the next section, we show a simple way to push this recovery even further with reselection.*

4.3.1 Enhanced Performance with Reselection

We can push the performance of `Caprese` with neuron reselection. For a sample, GRIFFIN and CATS calculate metrics ($\boldsymbol{s}$ and τ) to determine subsets of the FF block to use, but these metrics are fixed during the generation phase. After generating many tokens, these selection metrics can benefit from regular updates mid-generation.

For GRIFFIN, updating the metric $\boldsymbol{s}$ entails integrating the FF feature statistics of generated tokens into $\boldsymbol{s}$. While this can be done by re-running prefill on all tokens, there is a more efficient way. This can be done by passing in the generated tokens following the last reselection through the full model. As these tokens propagate through each layer, we find the selection metric for the generated tokens $\boldsymbol{s}_G$ and update corresponding KV pairs. This is similar to verification in speculative decoding [CBI+23, LKM23]. Since $\boldsymbol{s}$ and $\boldsymbol{s}_G$ are ℓ_2-norms along the token axis, we can define the updated metric as $\sqrt{(\boldsymbol{s} \odot \boldsymbol{s}) + (\boldsymbol{s}_G \odot \boldsymbol{s}_G)}$ and use that to reselect different subsets of the FF block to use. *This setup updates the pruned layers yet avoids prefill for all tokens.* Table 3 shows a clear benefit of reselection (even if infrequent) in `Caprese` by pushing the performance much closer to the full model.

Reselection is also possible with CATS but requires recomputing prefill for all tokens. The parameter to update is the hard thresholding parameter τ, but because this requires finding the desired percentile of intermediate values in the FF block, we would need to access to all values, which likely cannot be fully stored in memory. Consequently, prefill will need to be redone anytime we want to update τ. Additionally, since τ represents a cutoff for a desired percentile (e.g., median), it is fairly robust to new observations. Given this and the lack of computational incentive, we focus primarily on reselection for GRIFFIN.

4.3.2 Effect on Natural Response Length

`Caprese` implicitly encourages brevity, often even producing shorter responses than from the full thinking model. Intriguingly, this behavior arises despite any enforcement or regularization on response lengths anywhere during training or inference (except for cutting off generation at 32K tokens). Shown in Figure 4 with MATH-500, the shortest mean response length for all problem difficulties and subjects is either the full model or `Caprese`. Meanwhile, CATS consistently outputs the longest responses, averaging roughly 1K more across every sample. It is also interesting to note that with increasing problem difficulty, all models and methods naturally allocate more inference tokens towards answering the question. See Appendix C for similar observations with GRIFFIN.

Given CATS and `Caprese` (CATS) achieve similar MATH-500 accuracies to the full DeepSeek-R1-Distill-Qwen 7B and 14B models, the ability of `Caprese` to cut down the lengthy responses of CATS down to the response lengths of the original model or shorter without compromising performance is a direct memory and latency benefit. As we will see in the next section, we can also reduce the per-token latency for further computational savings.

4.4 Efficiency

`Caprese` reduces end-to-end generation latency. Table 4 shows the latencies of different setups and models. `Caprese` cuts latency by >11% when generating 2048 or fewer tokens from a prompt of 2048 tokens using

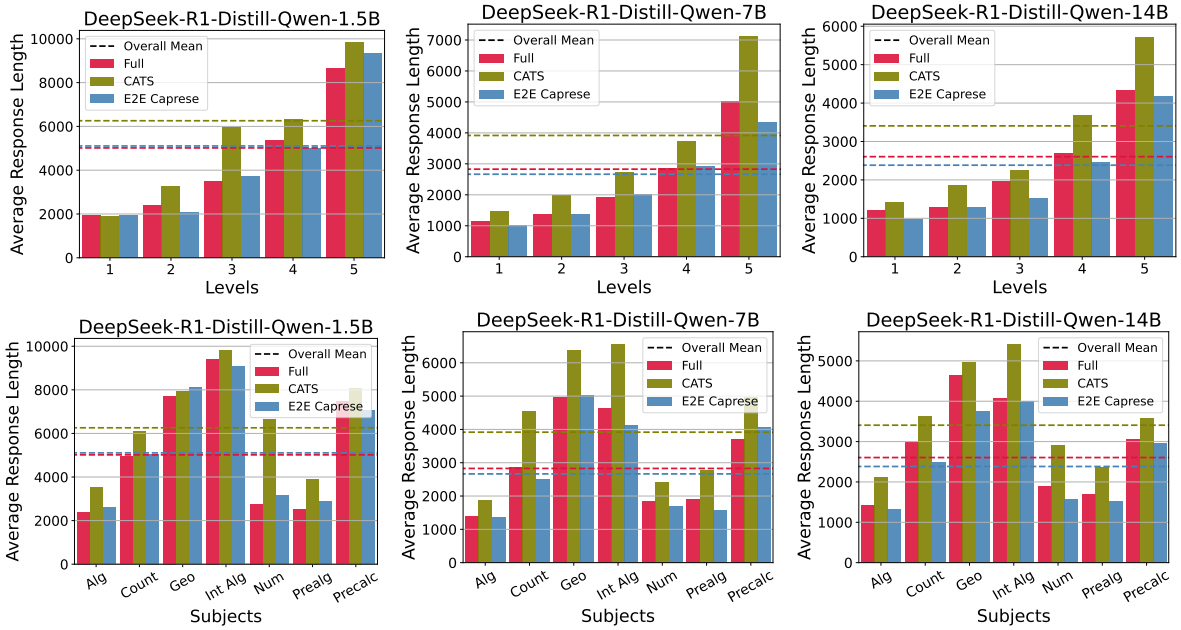

Figure 4: Average number of response tokens for MATH-500 queries across different problem difficulties (top) and subjects (bottom). The subjects are algebra, counting & probability, geometry, intermediate algebra, number theory, prealgebra, and precalculus. The global averages are indicated by the dashed lines. Sparsity is set at 50%.

Model	Setup	Full	GRIFFIN	Caprese
Qwen 2.5 14B	2048+256	12.8	11.1 (−13.3%)	11.3 (−11.7%)
	2048+2048	113.3	98.6 (−13.0%)	100.1 (−11.7%)
	2048+8192	661.9	603.2 (−8.9%)	603.7 (−8.8%)
Gemma 2 9B	2048+256	9.6	8.4 (−12.5%)	8.5 (−11.5%)
	2048+2048	84.4	74.5 (−11.7%)	75.1 (−11.0%)
	2048+8192	483.6	444.4 (−8.1%)	445.8 (−7.8%)

Table 4: End-to-end generation latency (s) for GRIFFIN and `Caprese` (GRIFFIN). For the "Setup" column, $P + G$ indicates input and generation lengths of P and G tokens, respectively. Percentages in parentheses show the reduction in latency. As before, GRIFFIN is applied to the first half of the model, sparsity is set at 50%, and $r = 256$.

DeepSeek-R1-Distill-Qwen 14B (Qwen 2.5 14B architecture) or Gemma 2 9B. Latency improvements are smaller with generation length 8192 but stilln faster than the full model. Moreover, the latency differences between GRIFFIN and `Caprese` are fairly small, suggesting that `Caprese` has *minimal overhead*. Since we use GRIFFIN on half of each model, the latency reductions are smaller than [DCC24] recorded. Metrics were collected on an NVIDIA L40 GPU using BF16 precision and pre-allocated memory for the KV cache. Time to first/next token metrics are included in Appendix D.

5 Conclusion

To combat the inefficiency of the long and brittle generation process associated with math reasoning, we introduce `Caprese`, a highly performant and efficient method with strong math and language capabilities that is compatible with a broad class of efficient FF algorithms. In the future, it would be interesting to evaluate its benefit in other reasoning domains besides math, explore the use of input-dependent low-rank layers, and investigate the knowledge contained in these low-rank layers. `Caprese`, along with future developments, pushes towards the long-term goal of degradation-free efficient LLM inference.

Acknowledgements

The work of H. Dong is supported in part by the Wei Shen and Xuehong Zhang Presidential Fellowship at Carnegie Mellon University.

References

[ABH25] S. A. Aytes, J. Baek, and S. J. Hwang. Sketch-of-thought: Efficient llm reasoning with adaptive cognitive-inspired sketching, 2025.

[AZ25] D. Arora and A. Zanette. Training language models to reason efficiently. *arXiv preprint arXiv:2502.04463*, 2025.

[BJE+24] B. Brown, J. Juravsky, R. Ehrlich, R. Clark, Q. V. Le, C. Ré, and A. Mirhoseini. Large language monkeys: Scaling inference compute with repeated sampling. *arXiv preprint arXiv:2407.21787*, 2024.

[CBI+23] C. Chen, S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre, and J. Jumper. Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318*, 2023.

[CKB+21] K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, C. Hesse, and J. Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

[DCC23] H. Dong, B. Chen, and Y. Chi. Towards structured sparsity in transformers for efficient inference. In *Workshop on Efficient Systems for Foundation Models@ ICML2023*, 2023.

[DCC24] H. Dong, B. Chen, and Y. Chi. Prompt-prompted adaptive structured pruning for efficient llm generation. In *First Conference on Language Modeling*, 2024.

[DDZ+24] D. Dai, C. Deng, C. Zhao, R. Xu, H. Gao, D. Chen, J. Li, W. Zeng, X. Yu, Y. Wu, et al. Deepseekmoe: Towards ultimate expert specialization in mixture-of-experts language models. *arXiv preprint arXiv:2401.06066*, 2024.

[DJP+24] A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Yang, A. Fan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

[DLB+21] P. Dasigi, K. Lo, I. Beltagy, A. Cohan, N. A. Smith, and M. Gardner. A dataset of information-seeking questions and answers anchored in research papers. *arXiv preprint arXiv:2105.03011*, 2021.

[DLBZ22] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer. Llm. int8 (): 8-bit matrix multiplication for transformers at scale. *arXiv preprint arXiv:2208.07339*, 2022.

[DYZ+24] H. Dong, X. Yang, Z. Zhang, Z. Wang, Y. Chi, and B. Chen. Get more with less: Synthesizing recurrence with kv cache compression for efficient llm inference. In *Forty-first International Conference on Machine Learning*, 2024.

[FA23] E. Frantar and D. Alistarh. Massive language models can be accurately pruned in one-shot. *arXiv preprint arXiv:2301.00774*, 2023.

[Fac25] H. Face. Open r1: A fully open reproduction of deepseek-r1, January 2025.

[GSBL20] M. Geva, R. Schuster, J. Berant, and O. Levy. Transformer feed-forward layers are key-value memories. *arXiv preprint arXiv:2012.14913*, 2020.

[GTA+23] L. Gao, J. Tow, B. Abbasi, S. Biderman, S. Black, A. DiPofi, C. Foster, L. Golding, J. Hsu, A. Le Noac'h, H. Li, K. McDonell, N. Muennighoff, C. Ociepa, J. Phang, L. Reynolds, H. Schoelkopf, A. Skowron, L. Sutawika, E. Tang, A. Thite, B. Wang, K. Wang, and A. Zou. A framework for few-shot language model evaluation, 12 2023.

[GYZ+25] D. Guo, D. Yang, H. Zhang, J. Song, R. Zhang, R. Xu, Q. Zhu, S. Ma, P. Wang, X. Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

[HBK+21] D. Hendrycks, C. Burns, S. Kadavath, A. Arora, S. Basart, E. Tang, D. Song, and J. Steinhardt. Measuring mathematical problem solving with the math dataset. *arXiv preprint arXiv:2103.03874*, 2021.

[HBM+22] J. Hoffmann, S. Borgeaud, A. Mensch, E. Buchatskaya, T. Cai, E. Rutherford, D. d. L. Casas, L. A. Hendricks, J. Welbl, A. Clark, et al. Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556*, 2022.

[JJNH91] R. A. Jacobs, M. I. Jordan, S. J. Nowlan, and G. E. Hinton. Adaptive mixtures of local experts. *Neural computation*, 3(1):79–87, 1991.

[KMH+20] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.

[LDS89] Y. LeCun, J. Denker, and S. Solla. Optimal brain damage. *Advances in neural information processing systems*, 2, 1989.

[LKB+23] H. Lightman, V. Kosaraju, Y. Burda, H. Edwards, B. Baker, T. Lee, J. Leike, J. Schulman, I. Sutskever, and K. Cobbe. Let's verify step by step. *arXiv preprint arXiv:2305.20050*, 2023.

[LKM23] Y. Leviathan, M. Kalman, and Y. Matias. Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning*, pages 19274–19286. PMLR, 2023.

[LLZ+24] D. Lee, J.-Y. Lee, G. Zhang, M. Tiwari, and A. Mirhoseini. Cats: Contextually-aware thresholding for sparsity in large language models. *arXiv preprint arXiv:2404.08763*, 2024.

[LWD+23] Z. Liu, J. Wang, T. Dao, T. Zhou, B. Yuan, Z. Song, A. Shrivastava, C. Zhang, Y. Tian, C. Re, et al. Deja vu: Contextual sparsity for efficient llms at inference time. In *International Conference on Machine Learning*, pages 22137–22176. PMLR, 2023.

[LYB+22] Z. Li, C. You, S. Bhojanapalli, D. Li, A. S. Rawat, S. J. Reddi, K. Ye, F. Chern, F. Yu, R. Guo, et al. The lazy neuron phenomenon: On emergence of activation sparsity in transformers. In *The Eleventh International Conference on Learning Representations*, 2022.

[MFW23] X. Ma, G. Fang, and X. Wang. Llm-pruner: On the structural pruning of large language models. *Advances in neural information processing systems*, 36:21702–21720, 2023.

[MSE24] R. Manvi, A. Singh, and S. Ermon. Adaptive inference-time compute: Llms can predict if they can do better, even mid-generation. *arXiv preprint arXiv:2410.02725*, 2024.

[NRS+24] S. Nayab, G. Rossolini, M. Simoni, A. Saracino, G. Buttazzo, N. Manes, and F. Giacomelli. Concise thoughts: Impact of output length on llm reasoning and cost. *arXiv preprint arXiv:2407.19825*, 2024.

[QYS+25] Y. Qu, M. Y. Yang, A. Setlur, L. Tunstall, E. E. Beeching, R. Salakhutdinov, and A. Kumar. Optimizing test-time compute via meta reinforcement finetuning. In *Workshop on Reasoning and Planning for Large Language Models*, 2025.

[RCM19] S. Reddy, D. Chen, and C. D. Manning. Coqa: A conversational question answering challenge. *Transactions of the Association for Computational Linguistics*, 7:249–266, 2019.

[RG24] M. Renze and E. Guven. The benefits of a concise chain of thought on problem-solving in large language models. In *2024 2nd International Conference on Foundation and Large Language Models (FLLM)*, pages 476–483. IEEE, 2024.

[SHZ+24] H. Sun, M. Haider, R. Zhang, H. Yang, J. Qiu, M. Yin, M. Wang, P. Bartlett, and A. Zanette. Fast best-of-n decoding via speculative rejection. *arXiv preprint arXiv:2410.20290*, 2024.

[SLBK23] M. Sun, Z. Liu, A. Bair, and J. Z. Kolter. A simple and effective pruning approach for large language models. *arXiv preprint arXiv:2306.11695*, 2023.

[SLXK24] C. Snell, J. Lee, K. Xu, and A. Kumar. Scaling llm test-time compute optimally can be more effective than scaling model parameters. *arXiv preprint arXiv:2408.03314*, 2024.

[TRP+24] G. Team, M. Riviere, S. Pathak, P. G. Sessa, C. Hardin, S. Bhupatiraju, L. Hussenot, T. Mesnard, B. Shahriari, A. Ramé, et al. Gemma 2: Improving open language models at a practical size. *arXiv preprint arXiv:2408.00118*, 2024.

[VSP+17] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

[WSL+24] Y. Wu, Z. Sun, S. Li, S. Welleck, and Y. Yang. An empirical analysis of compute-optimal inference for problem-solving with language models. *arXiv preprint arXiv:2408.00724*, 2024.

[WWS+22] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. V. Le, D. Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837, 2022.

[XXZH25] S. Xu, W. Xie, L. Zhao, and P. He. Chain of draft: Thinking faster by writing less. *arXiv preprint arXiv:2502.18600*, 2025.

[YYZ+24] A. Yang, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Li, D. Liu, F. Huang, H. Wei, et al. Qwen2. 5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.

[ZCX+24] Y. Zhou, Z. Chen, Z. Xu, V. Lin, and B. Chen. Sirius: Contextual sparsity with correction for efficient llms. *arXiv preprint arXiv:2409.03856*, 2024.

[ZLL+21] Z. Zhang, Y. Lin, Z. Liu, P. Li, M. Sun, and J. Zhou. Moefication: Transformer feed-forward layers are mixtures of experts. *arXiv preprint arXiv:2110.01786*, 2021.

[ZRG+22] S. Zhang, S. Roller, N. Goyal, M. Artetxe, M. Chen, S. Chen, C. Dewan, M. Diab, X. Li, X. V. Lin, T. Mihaylov, M. Ott, S. Shleifer, K. Shuster, D. Simig, P. S. Koura, A. Sridhar, T. Wang, and L. Zettlemoyer. Opt: Open pre-trained transformer language models, 2022.

A Effect of Rank & Sparsity

Here, we ablate the relationship between varying ranks in `Caprese` and sparsity levels in CATS. Using Llama 3.2 1B Instruct, we show the test performance of MATH in Figure 5. The same training procedure and data are used as outlined in Section 3. For all ablated ranks, `Caprese` consistently outperforms pure CATS by a large margin when CATS performs poorly relative to the full model. With a couple of exceptions (perhaps due to randomness in the generation process), greater performance is correlated with higher rank.

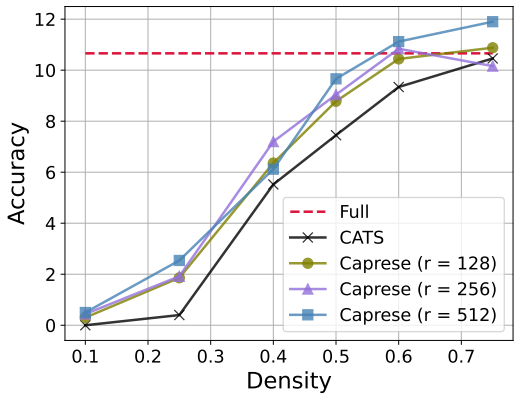

Figure 5: Llama 3.2 1B Instruct's performance on MATH with varying densities of CATS and ranks in `Caprese` with end-to-end training.

B `Caprese` Parameters

`Caprese` has a tiny parameter footprint. In Figure 6, we see that `Caprese` adds roughly 1% new parameters relative to the full model with a trend downwards as model size increases.

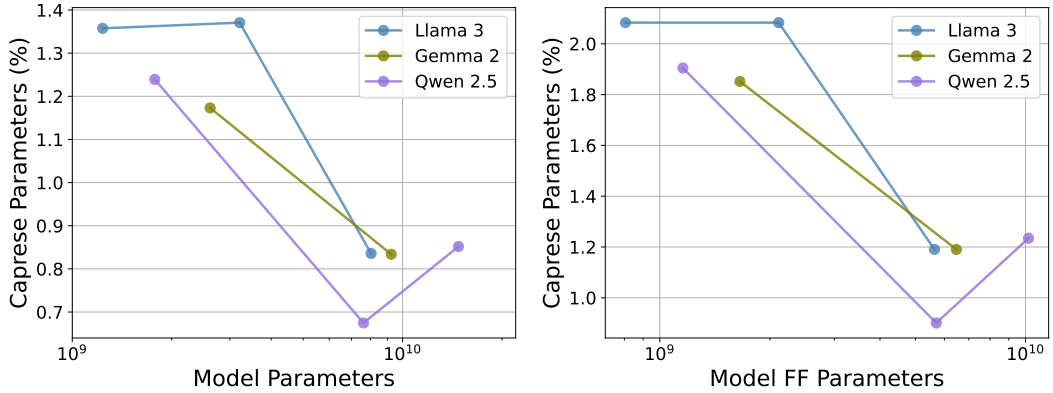

Figure 6: The percent of new parameters that `Caprese` ($r = 256$) adds relative to the entire model (left) and relative to only the FF parameters (right) for the Llama 3, Gemma 2, and Qwen 2.5 model families.

C GRIFFIN Response Lengths

Complementing Figure 4 for CATS, we show the natural response lengths to MATH-500 samples in Figure 7. Here, we see the similar observations as in Section 4.3.2, though the difference between `Caprese` (GRIFFIN)

and the full model is slightly larger but decreasing with model size. Even so, response lengths of `Caprese` is still significantly shorter than GRIFFIN.

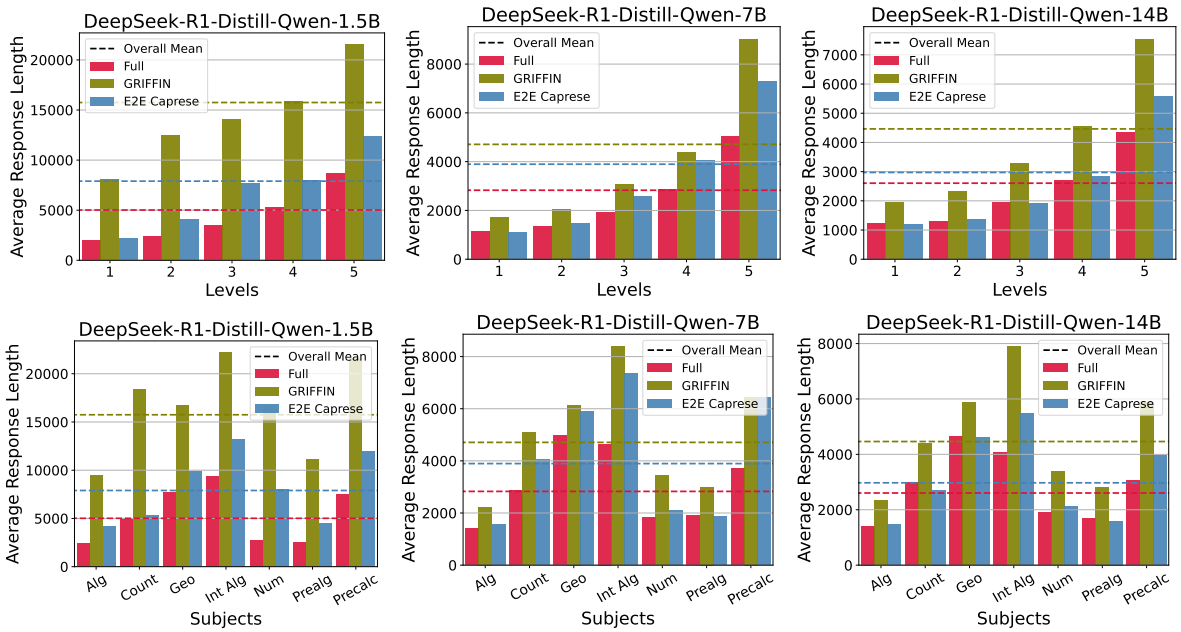

Figure 7: Average number of response tokens for MATH-500 queries across different problem difficulties (top) and subjects (bottom) when using GRIFFIN. The subjects are algebra, counting & probability, geometry, intermediate algebra, number theory, prealgebra, and precalculus. The global averages are indicated by the dashed lines. Sparsity is set at 50%.

D Additional Efficiency Metrics

Using the same setup from Section 4.4, we also report the average time to first token (TTFT) and time to next token (TTNT) in Table 5. The GRIFFIN and `Caprese` add a small overhead during the prefill phase but generally improve TTNT by 7ms and 5ms for Qwen 2.5 14B and Gemma 2 9B, respectively. There is hardly any difference in TTFT and TTNT between GRIFFIN and `Caprese`, reinforcing the fact that `Caprese` adds negligible overhead on top of GRIFFIN.

Model	Setup	Avg. Time to First Token			Avg. Time to Next Token		
		Full	GRIFFIN	Caprese	Full	GRIFFIN	Caprese
Qwen 2.5 14B	2048+256	0.64	0.67	0.67	0.048	0.041	0.042
	2048+2048	0.67	0.68	0.68	0.055	0.048	0.049
	2048+8192	0.77	0.80	0.80	0.081	0.074	0.074
Gemma 2 9B	2048+256	0.42	0.46	0.46	0.036	0.031	0.032
	2048+2048	0.45	0.49	0.50	0.041	0.036	0.036
	2048+8192	0.50	0.53	0.53	0.059	0.054	0.054

Table 5: Time to first and next token (s) for GRIFFIN and `Caprese` (GRIFFIN). For the "Setup" column, $P+G$ indicates input and generation lengths of P and G tokens, respectively. As before, GRIFFIN is applied to the first half of the model, sparsity is set at 50%, and $r = 256$.

E Training Hyperparameters

Table 6 lists the hyperparameter settings for training `Caprese` layers. The E2E learning rates lie in the interval [4e-6, 2e-4], where larger models tend to learn better with smaller learning rates.

	Layer-wise	End-to-end
Optimizer	Adam	Adam
Learning rate	1e-3	[4e-6, 2e-4] (varies)
Batch size	128	16
Epochs	20	3
Training samples	2e5	2e5
Scheduler	Linear	Linear
Warmup	2%	2%

Table 6: `Caprese` layer-wise and E2E training hyperparameters.